# Homework 7: Recursion and Classes

## Total Points: 100 (Correctness and Style) + 3 EC (Checkpoint)

**Due Dates (SD time):**

- **Entire Assignment: Monday, May 19th, 11:59pm**
- **Checkpoint (read below): Saturday, May 17th, 11:59pm**

## Starter Files

Download hw07.zip Inside the archive, you will find starter files for the questions of this homework. You cannot import anything to solve the problems.

## IMPORTANT: Coding and Docstring Style

This is a reminder that your code style will be graded. Here are a few useful links:

**Link to the requirements (I moved it to the separate page from now on).**

Style Guide on Course Website

Style Guide Examples

## Testing

At any point of the homework, use one of the following command to test your work:

```
>>> python3 -m doctest hw07.py
>>> py -m doctest hw07.py
>>> python -m doctest hw07.py
```

## Checkpoint Submission

### Due Date: Saturday, **May** 17th, 11:59pm (SD time)

You can earn **3 points extra credit** by submitting the checkpoint by the due date above. In the checkpoint submission, you should complete:

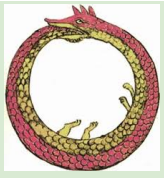
- Question 1
- Question 2
- Question 3

and submit the hw07.py file to gradescope.

Checkpoint submission is graded by completion, which means you can get full points if your code can pass some simple sanity check (no tests against edge cases). Note that in your final submission, you should still submit these questions, and you may modify your implementation if you noticed any errors.

## Required Questions

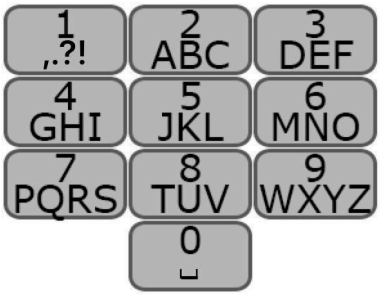
1. **DO NOT IMPORT ANY PACKAGES.**
2. Please add your own doctests (**at least three**) as the given doctests are not sufficient. **You will be graded on the doctests**.
    a. You do not need to add doctests for each getter/constructor. Instead, create at least 3 new objects of each class (Phone, Song, and Playlist) in order to test constructors and getters as a whole.
    b. For every non-getter method, add at least 3 new doctests.
    c. It is ok if the doctests go over 80 characters. You will not be marked off for this.
    d. When adding doctests, add them to the bottom of the doctests_go_here() docstring. There is a comment indicating where you should add your doctests. This will make it easier for graders to see your new doctests so you don't get marked off.
3. Docstrings are provided for Questions 3-5



## Recursion will not leave you any more

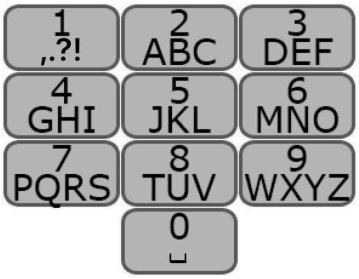
## Question 1

It’s time to think about the products your company will sell. The first idea is to enter the market by selling phones of all types, including old-fashioned ones. A customer can only use 10 number buttons to type. To make it work, follow the rules below. One can type:

| | |
|---|---|
| • ',.?!' with number 1<br>• 'abc' with number 2<br>• 'def' with number 3<br>• 'ghi' with number 4<br>• 'jkl' with number 5<br>• 'mno' with number 6<br>• 'pqrs' with number 7<br>• 'tuv' with number 8<br>• 'wxyz' with number 9<br>• spaces with '0' |   |

Write a **recursive** function (no loops/LC/Map/Filters) that takes in a string and returns the sequence of digits you need to press on your phone, as a string. Your function should be case *insensitive*. **You can assume all inputs are valid.**

**For example:**

| Input | Output | |
|---|---|---|
| 'Dragon' | '372466'<br><br>**Reason:**<br>- D corresponds to 3<br>- r corresponds to 7<br>- a corresponds to 2<br>- … |   |

```python
def type_with_number(message):
    """
    >>> type_with_number('Welcome to Beijing!')
    '9352663086023454641'
    >>> type_with_number('I miss my laptop.')
    '40647706905278671'
    >>> type_with_number('!!??..  ,,')
    '1111110011'
    """
```

## Question 2

This question requires you to write a **recursive** function that creates palindromes.

**Note**: Palindrome is a string that reads the same backward as forward: *"Was it a car or a cat I saw?"*

This function takes two parameters:

- `start` (positive integer): element to start with.
- `stop` (positive integer): element at the center of the palindrome.

Your function should create a palindrome and return it as a string. Check doctest for examples.

```
def make_palindrome(start, stop):
    """
    >>> make_palindrome(1, 1)
    '1'
    >>> make_palindrome(3, 5)
    '34543'
    >>> make_palindrome(5, 2)
    '5432345'
    """
```

## Question 3

Create a class called `Phone` that represents a Phone object. There are 3 instance attributes (properties) passed in when initialized. The brand of the phone (string), its maximum battery capacity (integer representing mAh), and its storage capacity (integer representing MBs).

There are 5 more attributes that you must derive from the instance attributes:

1. The **charge** of the phone will be half of its maximum battery capacity (keep charge as an integer, use floor divide //).
2. The **drain_rate** represents how much charge the phone uses per minute and will be determined by the phone's brand:
    - 'Apple' will have a drain_rate of 10.
    - 'OnePlus' will have a drain_rate of 12.
    - 'Samsung' will have a drain_rate of 8.
    - All other brands will have a drain_rate of 15.
3. The **charge_rate** represents how much the phone charges per minute when charging. The charge_rate will be 20, across all devices.
4. The **num_apps** represents how many applications are installed on the device. Out of the box, no apps will be installed.
5. The **apps** represent a set (use Python's `set()`) of the names of the applications that are installed on the device. Since there are no apps installed out of the box, it will be initialized as an empty set.

After the constructor, you must implement 3 methods.

**use(self, minutes)** will update the phone's charge based on how many minutes it is used and what the phone's `drain_rate` is. In the case where we run out of charge, you will return the string '`Out of charge`'. You should make sure that the charge is not negative in the case where the charge is drained. In normal use cases, you do not need to return anything.

**recharge(self, minutes)** will update the phone's charge based on how many minutes it is recharged. In the case where we overcharge, you should make sure that the charge does not exceed the battery's maximum capacity.

**install(self, app_size, app_name)** will install an app with the given app_name (string) and app_size (integer representing MB). You must consider the following cases before you can install the app.

1. An app cannot be installed if we have no charge
    - Return '`Out of charge`'
2. An app cannot be installed if we do not have sufficient storage. We can install apps up until we have 0MB of storage left
    - Return '`Not enough storage`'
3. An app cannot be installed if it is already installed. We base this off of the instance attribute **apps**
    - Return '`App already installed`'

Once we have dealt with these cases, we can install our app and return '`App installed`'. Make sure that you are updating storage, num_apps, and apps.

*Use a starter for the doctests and skeleton of the class.*

## OOP Introduction

Music streaming has become the most popular medium to listen to music in recent years, as people started to shift away from CDs and vinyl records. In this assignment, we are going to create a simplified version of a music streaming platform such as `Spotify`. We recommend you to read the entire writeup before starting to code, as you will have a better understanding of how the different methods and attributes will work together to create the functionality of the classes.

When we write functions we try to make sure that every function does exactly one job. A similar approach works with classes. We want to create a class that describes the functionality of a particular object and then use these classes (objects) to send messages to each other.

To get a better understanding of Object-Oriented Programming in Python, we are going to implement two Python classes: **Song** and **Playlist**

## OOP Diagram

| Song | Playlist |
|---|---|
| **Instance Fields:** | **Instance Fields:** |
| name (str) | title (str) |
| length (float) | user (str) |
| album (str) | **Instance Attributes (not passed in):** |
| artist (str) | songs (list of Songs) |
| streams (int) | |
| **Class Attributes:** | |
| platform (str) | |
| **Constructor:** | **Constructor:** |
| __init__(self, name, length, …) | __init__(self, title, user) |
| **Getters:** | **Getters:** |
| get_name(self) | get_title(self) |
| get_length(self) | get_user(self) |
| get_album(self) | get_songs(self) |
| get_artist(self) | get_total_streams(self) |
| get_streams(self) | get_total_length(self) |
| | get_most_played_song(self) |
| **String Representation:** | **String Representation:** |
| __str__(self) | __str__(self) |
| **Core Functionality:** | **Core Functionality:** |
| listen(self) | add_song(self, song) |
| add_to_playlist(self, playlist) | remove_song(self, song) |
| | sort_songs(self, sort_by) |
| | play(self) |
| | combine_playlists(self, other_playlist) |

## Part 1: Constructor & methods for Song class

# Class "Song"

**Question 4.1** : Instance attributes of **Song** class

A. **name (str):** Given by the constructor's argument. After initialization, `name` indicates the name of the song. The name of a song cannot be an empty string.
B. **length (float):** Given by the constructor's argument. After initialization, `length` indicates the length of the song in minutes. The length of a song must be positive.
C. **album (str):** Given by the constructor's argument. After initialization, `album` indicates the name of the album of the song. The name of an album cannot be an empty string.
D. **artist (str):** Given by the constructor's argument. After initialization, `artist` indicates the name of the artist who created the song. The name of an artist cannot be an empty string.
E. **streams (int):** Given by the constructor's argument. After initialization, `streams` indicates the number of streams the song has. The number of streams cannot be negative.

Class attribute of **Song** class

A. **platform (str)**: Class attribute that contains the name of the platform the song is hosted on. The platform we will be using is 'Spotify'

**Question 4.2** : Methods of **Song** class

A. **Constructor (__init__):**
The constructor has parameters described in Question 4.1 and the attributes names are the same as these parameters. You should write assert statements according to each attribute description above.

B. **Getters:**
   a. **get_name(self)**
   Getter method that returns the name of the Song. This method creates the data abstraction.
   Example: return 'Drake' if the Song's name is 'Drake'
   b. **get_length(self)**
   Getter method that returns the length. This method creates the data abstraction.
   c. **get_album(self)**

Getter method that returns the album. This method creates the data abstraction.

d. **get_artist(self)**
   Getter method that returns the artist. This method creates the data abstraction.
e. **get_streams(self)**
   Getter method that returns the streams. This method creates the data abstraction.

*Note*: All of these getter methods only require a single line

### C. String representation method

a. **__str__ (self)**
   In order to print a string representation of an object we need to write a special method (will be covered in lectures later). This method enables us to use print statements on instances of Song. Implement the special method using the getter methods created above (part 4.2B).

   <u>The format of the string should be (if printed, without parentheses, on a single line):</u>
   '(song name)' by (artist) on '(album)' is (song length) minutes long with (# of song streams) streams

### D. Core Functionality

a. **listen(self):**
   Method is used when someone listens to a song, returning a string of the format (without parentheses)
   "Listening to '(song name)' by (artist name)"
   It should also increment the streams attribute for the Song.
b. **add_to_playlist(self, playlist)**
   Adds the Song to the given Playlist instance. You will see below that the Playlist class has an add_song() method — think of the way to use these methods together to write less repetitive code.

   **Requirements**: Write an assert statement for this method.
   **Hint**: try to see what the type of the parameter should be or what is the object being passed.

## Part 2: Constructor & methods for Playlist class

# Class "Playlist"

**Question 5.1** : Instance attributes of **Playlist** class

A. **title (str):** Given by the constructor's argument. After initialization, `title` indicates the title of the playlist. The title of a playlist cannot be an empty string.
B. **user (str):** Given by the constructor's argument. After initialization, `user` indicates the username of the user who created the playlist. The username cannot be an empty string.
C. **songs (list):** NOT provided by the constructor's argument. You should initialize this attribute by yourself. `songs` is a list of Song objects, which is how the Songs in the Playlist will be stored. After initialization, `songs` should **not** have any Songs in it.

**Question 5.2** : Methods of **Playlist** class

**A. Constructor (__init__):**
The constructor has parameters described in Question 2.1 and the attributes names are the same as these parameters. You should write assert statements according to each attribute description above (excluding songs).

**B. Getters:**

**a. get_title(self)**
Getter method that returns the title of the Playlist. This method creates the data abstraction.
Example: return 'Dope Beats' if the Playlist's title is 'Dope Beats'

**b. get_user(self)**
Getter method that returns the user of the Playlist. This method creates the data abstraction.

**c. get_songs(self)**
Getter method that returns the songs of the Playlist. This method creates the data abstraction.

**Note**: All of these getter methods only require a single line.

**C. String representation method**

**a. __str__ (self)**
In order to print a string representation of an object we need to write a special method (will be covered in lectures later). This method enables us to use print statements on instances of Playlist. Implement the special method using the getter methods created above (part 5.2B).

<u>The format of the string should be (if printed, without parentheses, on a single line):</u>
Playlist '(title)' by (user) has (# of Songs in songs) songs

**D. Core Functionality**

**a. add_song(self, song):**

Add song to the playlist. If the song is already in the playlist, you should not add it again and instead return False. If you are able to add it to the playlist, do it and then return True.
**Requirements:** assert statements

**b. remove_song(self, song):**
Remove song from the playlist. If successful, return True. If not, return False.
**Requirements:** assert statements

**c. sort_songs(self, sort_by):**
Sort the songs in the playlist according to `sort_by` in ascending order. `sort_by` should be the name of one of the attributes of Song, as a string. For example, if sort_by is 'length', we should sort the songs based on their length attributes in ascending order. This method should mutate the songs attribute without returning anything.
*Note*: The method getattr() may be helpful
**Requirements:** assert statements

**d. get_total_streams(self):**
Return the total amount of streams of the songs in the playlist using one-line list comprehension.
**Requirements:** one-line list comprehension

**e. get_total_length(self):**
Return the total length of the playlist using 1 line list comprehension.
**Requirements:** one-line list comprehension

**f. play(self):**
Plays every song in the playlist once. You may want to use the method `listen()` from the Song class. You should return a string that contains information on all the songs played, with each song being separated by a newline. If the playlist is empty, return the string "Empty"

Example (if printed, without parentheses):
Listening to '(song1 name)' by (artist1 name)
Listening to '(song2 name)' by (artist2 name)

song1 and song2 represent different songs in the playlist. If there are more songs in the playlist, more lines should be included in the returned string.

**g. combine_playlists(self, other_playlist):**
Given another playlist, add all songs from other_playlist to the current playlist by using a list comprehension. If all songs from other_playlist are added successfully, return True. If not (remember that we cannot have

duplicate songs in the same playlist), return the number of songs that were not added to the current playlist. You may want to use previously defined methods in this class.

**Requirements: list comprehension only (no explicit for/while loops)**, assert statements

**h. get_most_played_song(self):**

Return the name of the most played song (largest amount of streams). You may want to use previously defined methods. You can assume that there will not be any ties. Be careful if you use sort_songs since it will modify the songs attribute. You may want to use a different implementation than what you did in sort_songs, since we want to find the song with the most streams without modifying the songs attribute.

- If there are no songs in the Playlist, this method should return an empty string.

**Requirement**: Write assert statements for all methods that have additional parameters
(add_song, remove_song, sort_songs, combine_playlists)

## Submission

By the end of this homework, you should have submitted the homework via Gradescope. You may submit more than once before the deadline; only the final submission will be graded. Stay on the submission page until the results show, to make sure that your submission is successfully processed by the Autograder. If your submission fails, the tests are not able to run and you would receive a 0.